

Arithmetic Circuits

Arithmetic Circuits

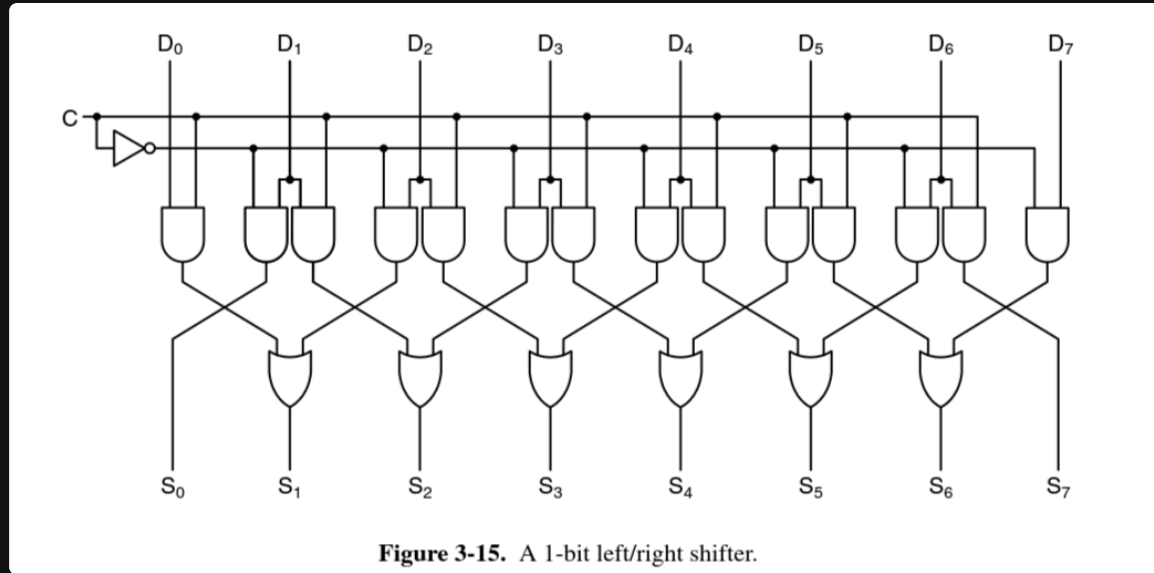
Arithmetic circuits are combinational circuits whose job is to perform *mathematical operations*

They take one or more binary words as input and produce a binary word as output, all **without any memory** of past inputs.

The circuits in this section, *shifters*, *adders*, and *the arithmetic logic unit* (ALU), are the components that a CPU combines to carry out the arithmetic and logic instructions in its instruction set.

Shifters

A shifter is a combinational circuit that moves the bits of a binary word left or right by some number of positions. An 8-bit shifter, for example:



- takes an 8-bit input and a control signal that selects the shift direction, and
- produces an 8-bit output in which every bit has been moved one or more places relative to its original position.

Shifters

There are two common kinds of shifts.

- A **logical shift** moves bits left or right and fills the vacated positions with zeros;

A *left* logical shift multiplies an unsigned number by two for each position shifted, and

A *right* logical shift divides it by two.

- An **arithmetic shift** behaves the same way when shifting left,

But when *shifting right* it fills the vacated most-significant bits with copies of the sign bit rather than zeros, so that a *two's-complement signed number* keeps its sign when divided by powers of two.

Two's Complement (quick review)

Two's complement is the standard way computers represent signed integers in binary.

In an n -bit two's-complement number, the most significant bit is the **sign bit**:

- it is 0 for non-negative values and 1 for negative values, and unlike sign-magnitude representation, that bit also carries ordinary place value Worth -2^{n-1} .

To negate a number in two's complement, invert every bit (take the one's complement) and then add 1;

For example,

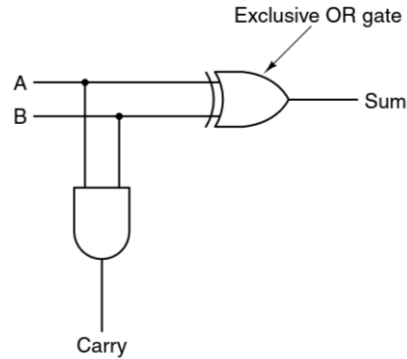
- in 4 bits, `0011`
- inverts to `1100` and
- adding 1 gives `1101` ,
- which represents -3 .

Two's complement is popular because ordinary binary addition works unmodified for both positive and negative numbers, and there is only one representation of zero.

Adders

Every computer needs to add binary numbers, and this is done with a circuit called an adder.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



And it produces

- a sum bit $S = A \text{ XOR } B$ and
- a carry-out bit $C = A \text{ AND } B$.

The half adder's limitation is that it has no way to accept a carry-in from a previous, less-significant bit position, so it cannot be chained with other adders to add multi-bit numbers.

The simplest building block is the **half adder**, which adds two single bits,

A and B ,

Full Adder

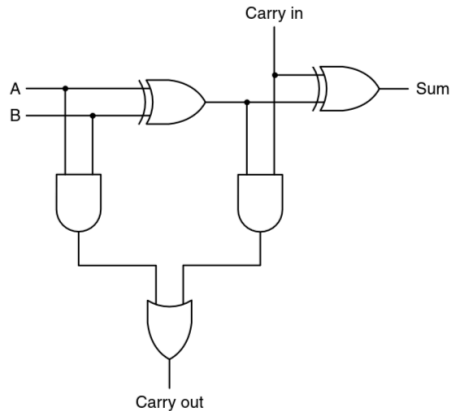
The **full adder** solves this problem by taking three inputs

- A ,
- B , and
- a carry-in C_{in}

And producing a sum bit and a carry-out bit that accounts for all three inputs.

Because it accepts a carry-in, full adders can be connected in a chain to add numbers wider than one bit.

A	B	Carry in	Sum	Carry out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Ripple Adder

To add two 16-bit words sixteen full adders are connected so that the carry-out of each bit position becomes the carry-in of the next more significant position,

With the carry-in of the least significant (rightmost) bit tied to 0.

This structure is called a ripple-carry adder, because the carry signal must propagate, or "ripple," through every stage before the final, most-significant sum bit is guaranteed to be correct.

The drawback of a ripple-carry adder is speed: its worst-case delay grows linearly with the number of bits, since the last stage cannot produce a valid result until the carry has rippled all the way from the first stage.

Carry Select Adder

One way to speed this up is the **carry-select adder**.

Instead of waiting for the real carry to ripple in, a carry-select adder computes each half of the word twice in parallel

- once assuming the incoming carry is 0, and
- once assuming it is 1

And then uses a multiplexer to select the correct precomputed result as soon as the actual carry-in becomes available.

For example,

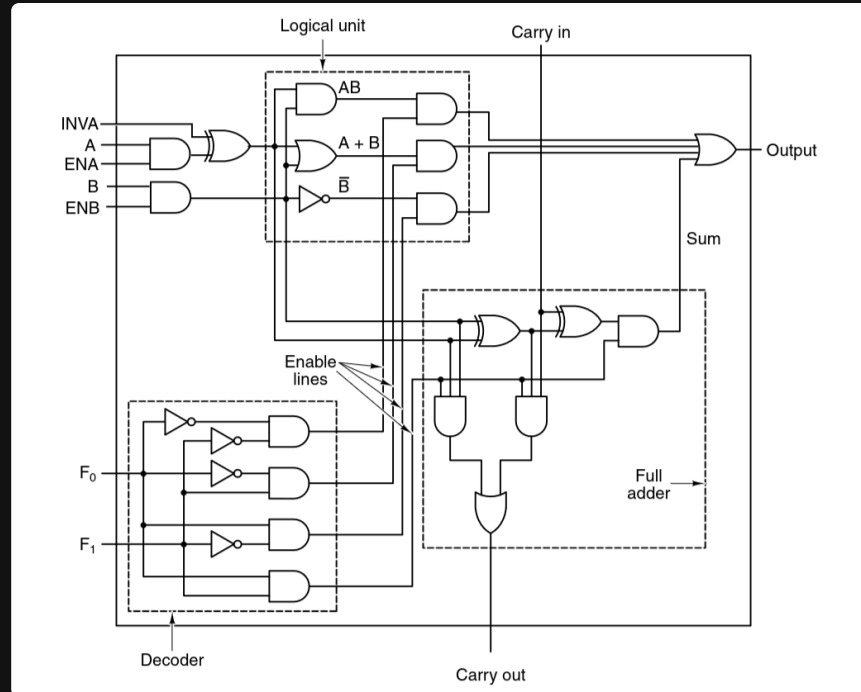
a 32-bit carry-select adder might be split into two 16-bit halves:

- the lower half computes its sum and carry-out normally,
- while the upper half is computed twice in parallel,

The moment the lower half's carry-out is known, a multiplexer immediately selects the correct upper-half result instead of waiting for a ripple through all 32 bits. This trades extra hardware for a significant reduction in worst-case delay.

ALU

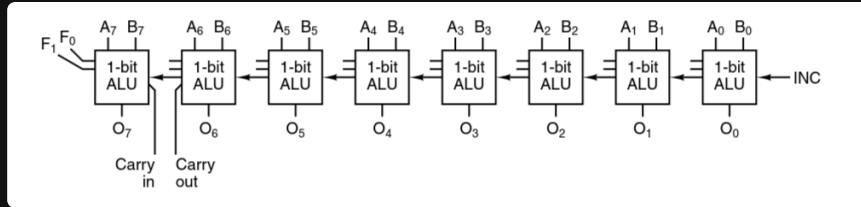
Most computers contain a single circuit block, the **arithmetic logic unit (ALU)**, that can perform several operations on two machine words



ALU

Typically AND, OR, and addition (and often subtraction as well), selecting which operation to perform based on control (select) lines.

A typical simple ALU can compute $A \text{ AND } B$, $A \text{ OR } B$, $\text{NOT } B$, or $A + B$, with the desired function chosen by its select inputs.



An n-bit ALU is built from n identical single-bit ALU "slices" chained together like a ripple-carry adder, with each slice handling one bit position and passing its carry to the next.

So, to build an ALU that operates on 16-bit words, a designer connects sixteen 1-bit ALU slices in a row, with each slice's carry-out feeding the carry-in of its neighbor.